Year: 2026-2027 Sem V-NEP

Thadomal Shahani Engineering College

Department of Artificial Intelligence & Data Science

Course: Agile Software Development & DevOps Lab (Course Code: 2165119)

Class: T2 Date of Submission: 15 October 2026

A Mini Project Report on

Student Performance Management System

A Student Marks, Attendance and Risk-Prediction Application Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027

Submitted by

Name	Roll No.
Gunjan Rupani	2401096
Krisha Samtani	2401099
Vanshika Singh	2401116

Under the guidance of

Prof. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled "**Student Performance Management System: A Student Marks, Attendance and Risk-Prediction Application Delivered Through an Agile and DevOps Pipeline**" is a bona fide work of **Gunjan Rupani (2401096)**, **Krisha Samtani (2401099)** and **Vanshika Singh (2401116)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide Head of Department

Date: 15 October 2026 Place: Mumbai

ABSTRACT

Teachers usually record marks and attendance in separate spreadsheets, so a student who is slipping is noticed only after the examination result is out. The Student Performance Management System (SPMS) brings students, courses, marks and attendance into one web application. It calculates each student's average and grade, flags every student whose average is below 50 or whose attendance is below 75 % as "at risk", and uses a scikit-learn regression model to estimate each student's chance of failing the final examination. A "What-if lab" lets a teacher change marks or attendance and see how the prediction moves, and a printable performance report can be produced for every student.

The application was used as the vehicle for applying the experiments of the Agile Software Development & DevOps Lab to one product. The work was planned in Jira as a Scrum project with five epics and ten stories, all of which reached Done; the source code is hosted on GitHub and connected to the Jira space. The Flask backend is containerised with Docker, and Docker Compose starts it together with Prometheus and Grafana as one stack. A Jenkins pipeline builds the project and deploys it; its last build (#10) is stable and successful. Prometheus scrapes the backend's /metrics endpoint and reports the target as UP, and a Grafana panel plots the availability of the service over time.

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Prometheus, Grafana, Flask, React, scikit-learn

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–5 and 7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

In most colleges the marks of a student are kept by one teacher and the attendance by another, usually in separate spreadsheets. Nobody sees the two together, and a student who is missing lectures and scoring poorly in class tests is typically identified only after failing the final examination, when it is too late to help. Teachers also spend time every term recalculating averages, grades and lists of defaulters by hand.

The Agile Software Development & DevOps Lab introduced, through separate experiments, the practices used to deliver software reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was performed on a small sample workload. This mini project applies them together to one realistic product so that their interaction, rather than each tool in isolation, becomes visible.

1.2 Problem Statement

To design, build and operate a web application that manages student records, marks and attendance, identifies students at risk and predicts the chance of failure, and to deliver it through an Agile, automated pipeline in which every change is planned in Jira, versioned in Git, built by Jenkins, deployed as containers and observed by monitoring tools.

1.3 Objectives

1. Add, search, filter, edit and delete students, with unique roll numbers.
2. Record marks (out of 100) by student, subject and exam, and compute the average and grade automatically.
3. Record attendance and flag students whose average is below 50 or attendance is below 75 % as at risk.
4. Predict each student's chance of failing the final examination with a regression model, and let the teacher test "what-if" changes.
5. Generate a performance report for each student and support import and export of data.
6. Plan the work as a Scrum project in Jira with epics, stories, a sprint and a Definition of Done.
7. Keep all source code in a GitHub repository linked to Jira.
8. Containerise the backend and run the complete stack with a single Docker Compose command.
9. Build and deploy automatically with Jenkins, and monitor the service with Prometheus and Grafana.

1.4 Scope

The system targets one institution or department and is used by teachers and administrators; students are the subjects of the records rather than users of the application. Authentication and multi-institution support are outside the scope of this version. The deployment target is a single host running Docker, which here is a developer laptop running Windows.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in SPMS	Section
1	Version control with Git	GitHub repository krishasamtani/Student-Performance-Management-System, branches, commits and merges, linked to Jira	5.1
2	Containerisation with Docker	Dockerfile for the Flask backend; container spms-backend running in Docker Desktop	5.2
3	Docker Compose	Three-service stack (backend, Prometheus, Grafana) started with docker compose up -d --build	5.3
4	Jenkins CI pipeline	Jenkins job Student-Performance-Management-System that builds and tests the project	5.4
5	Automated deployment	Successful Jenkins build #10 deploying the stack with Docker Compose	5.5
6	Agile lifecycle with Jira	Jira space SPMS: 5 epics, 10 stories, Sprint 2, timeline and reports	2
7	Monitoring	Prometheus target spms-backend scraped and Grafana panel on the up metric	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Teacher	Enter marks and attendance, find struggling students early	Marks, Attendance, Dashboard, Predictions
Administrator	Maintain students and courses, import and export data	Students, Courses, Import & export, Model

Table 2.1: User roles

2.2 Functional Requirements

- Students are added with a unique roll number, full name and course; the list can be searched, sorted, and filtered by course, grade and "at risk only".
- Marks are entered as a score from 0 to 100 for a student, exam and subject. Entering the same subject and exam again updates the score instead of creating a duplicate; scores can be edited or deleted at any time.
- The average of a student's marks is converted to a grade (A, B, …) and shown beside the attendance percentage.
- A student is "at risk" when the average is below 50 or the attendance is below 75 %. The dashboard lists the at-risk students with the reason.
- The dashboard shows the total number of students, the average score, the average attendance and the number of students at risk.
- The Predictions page ranks students by predicted chance of failing the final, using the regression model; the What-if lab recalculates the prediction for changed inputs; the Model page describes the trained model.
- A performance report can be opened for every student, and data can be imported and exported.

2.3 Non-Functional Requirements

- **Usability:** a single-page interface with a sidebar of Overview, Manage, Insights and Data sections.
- **Reliability:** the backend container restarts automatically and data persists in a mounted volume.
- **Portability:** the whole system starts with one docker compose command on any machine with Docker.
- **Observability:** the backend exposes a /metrics endpoint for Prometheus and its availability is graphed in Grafana.
- **Maintainability:** automated tests cover the API, the data handling and the machine-learning code.

2.4 Scrum Setup in Jira (Experiment 6)

The project was organised with Scrum, following the Agile project lifecycle of Experiment 6. A Jira space named **Student Performance Management System** (key **SPMS**) was created and connected to the GitHub repository. The work was divided into five epics, each broken into stories, and the stories were scheduled into a sprint named **SPMS Sprint 2**. Every work item was assigned to a team member and moved through the board to Done as it was completed. Figure 2.1 shows the Timeline view, in which each epic is a bar against the sprint and every epic shows the status "Done".

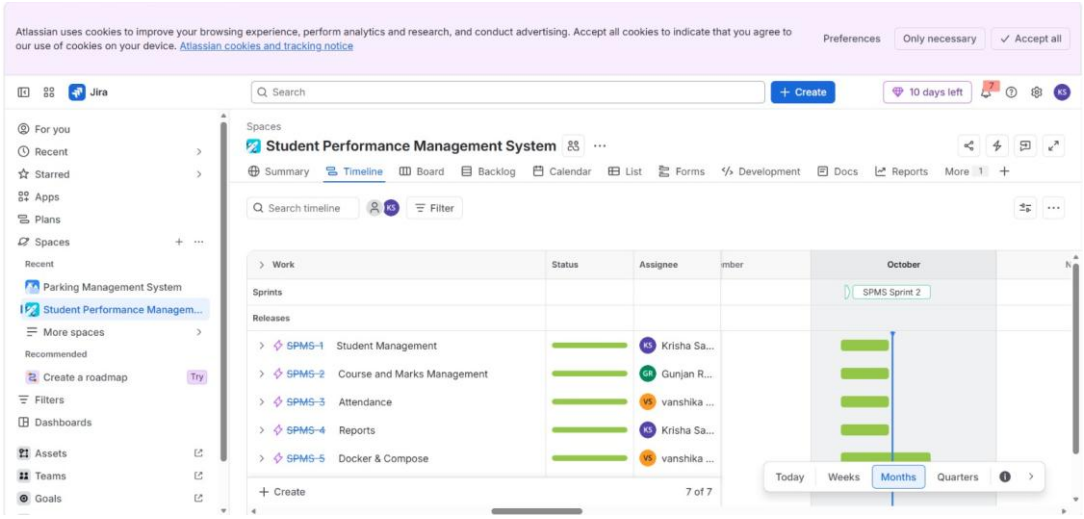


Figure 2.1: Jira Timeline of the SPMS space – five epics under Sprint 2

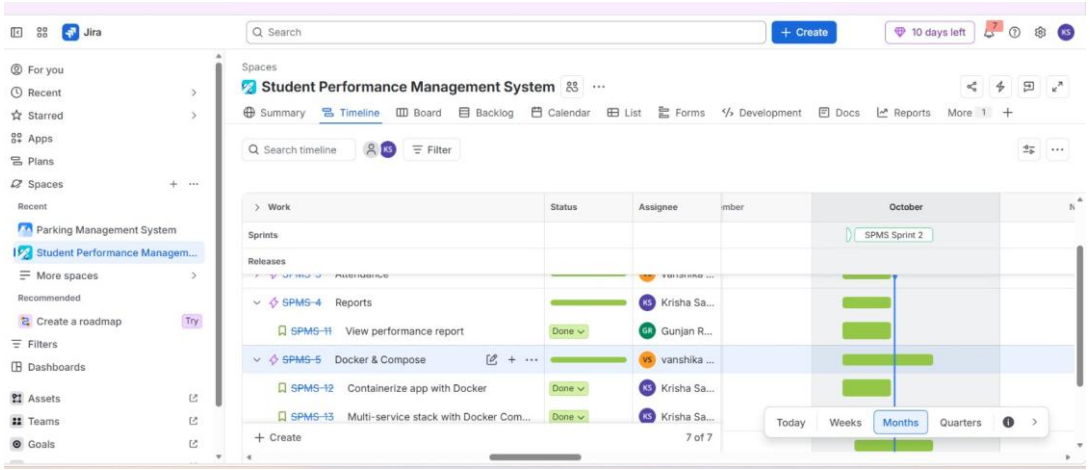


Figure 2.2: Timeline with the epics expanded to show their stories

2.5 Product Backlog

Key	Type	Summary	Owner	Status
SPMS-1	Epic	Student Management	Krisha Samtani	Done
SPMS-2	Epic	Course and Marks Management	Gunjan Rupani	Done
SPMS-3	Epic	Attendance	Vanshika Singh	Done
SPMS-4	Epic	Reports	Krisha Samtani	Done
SPMS-5	Epic	Docker & Compose	Vanshika Singh	Done

Key	Type	Summary	Owner	Status
SPMS-11	Story	View performance report (under SPMS-4)	Gunjan Rupani	Done
SPMS-12	Story	Containerize app with Docker (under SPMS-5)	Krisha Samtani	Done
SPMS-13	Story	Multi-service stack with Docker Compose (under SPMS-5)	Krisha Samtani	Done
SPMS-6 to 10, 14, 15	Story	Remaining stories of the five epics (student, course, marks, attendance and report features)	Team members	Done

Table 2.2: Product backlog of the SPMS Jira space

2.6 Sprint Report

Jira's Reports page (Figure 2.3) summarises the finished sprint. All 15 work items (5 epics and 10 stories) are in the Done status, and the items are shared between the team members. The same page also provides the creation trend and the cycle time of the work items.

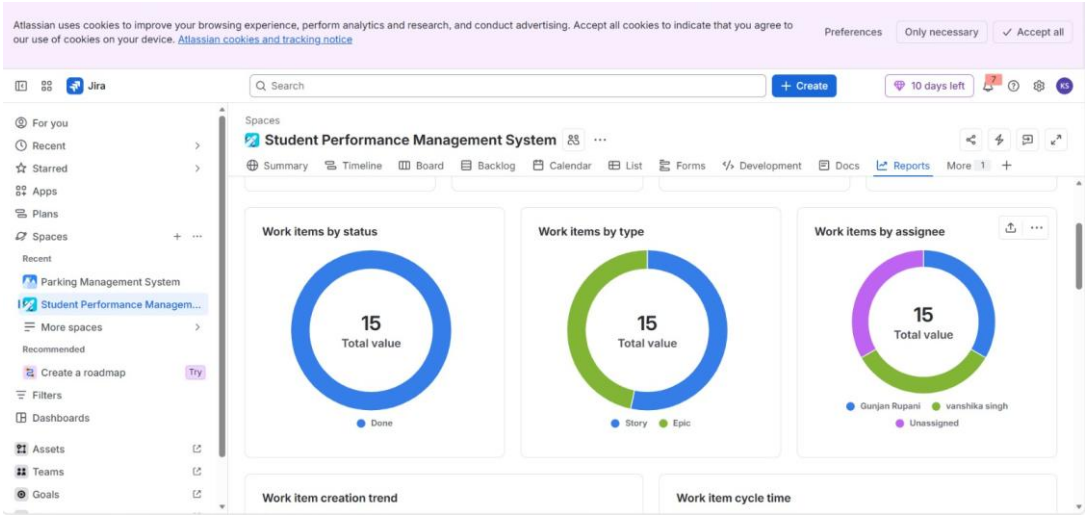


Figure 2.3: Jira reports – work items by status, type and assignee

2.7 Definition of Done

A work item was moved to Done only when all of the following held:

1. The code for the item was committed to the GitHub repository and visible from the Jira Development tab.
2. The automated tests passed.
3. The Jenkins build that contained the change reported SUCCESS.
4. The behaviour was verified in the running application and containers.

3. System Design

3.1 Architecture

SPMS is a three-tier web application. The browser loads a React single-page application (spms-frontend). The React code calls a REST API implemented in Flask (app.py), which reads and writes the student, course, marks and attendance records in a SQLite database file, spms.db. The machine-learning code in the ml folder is a scikit-learn model that the API uses to calculate failure predictions. The footer of the application states the stack: Flask, React and scikit-learn.

The backend runs in a Docker container named spms-backend, published on port 5000, with the database directory mounted from the host (./data to /app/data) so that records survive restarts and redeployments. Prometheus runs beside it in the same Compose network and scrapes the backend's /metrics endpoint; Grafana reads from Prometheus and draws the graphs. Outside the stack, GitHub holds the code, Jira holds the plan, and Jenkins builds the project and deploys the stack on the same host (Figure 3.1).

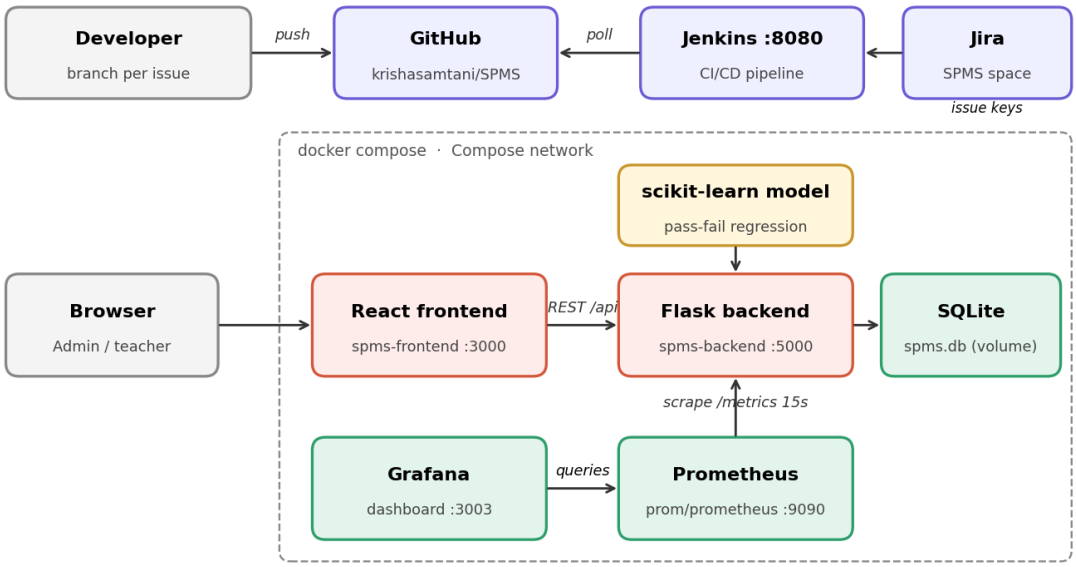


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	React (spms-frontend)	Component-based UI for the dashboard, tables, forms and charts; communicates with the API through api.js
Backend	Python, Flask (app.py)	Lightweight framework; the same language as the machine-learning code
Machine learning	scikit-learn (ml folder)	Regression model for the chance of failing the final
Database	SQLite (spms.db)	No separate server; one file persisted in a mounted volume

Layer	Technology	Reason for choice
Reports	report_pdf.py	Generates the per-student performance report
Containers	Docker, Docker Compose	Reproducible environment; one command starts the stack
CI/CD	Jenkins	Builds and deploys the project automatically
Planning / VCS	Jira, Git, GitHub	Agile tracking linked to the code repository
Monitoring	Prometheus, Grafana	Time-series metrics and dashboards

Table 3.1: Technology stack

3.3 Data Design

The database holds four kinds of record. A **student** has a unique roll number (for example 2401099), a full name and a course. A **course** groups students. A **mark** records a score from 0 to 100 for one student, one subject and one exam type, such as Mid-term; the pair of subject and exam is unique for a student, which is why entering it again updates the score. An **attendance** record stores the lectures held and attended for a student, from which the attendance percentage is derived. The average, grade, attendance percentage and risk status shown on the Students page are calculated from these records and are not stored.

3.4 Risk Rule and Prediction

Two mechanisms identify students who need help. The first is a transparent rule: a student is at risk when the average score is below 50 or the attendance is below 75 %. The second is statistical: a scikit-learn regression model is trained on the recorded data and returns a probability of failing the final examination. The dashboard shows the students with the highest probability in a “Predicted to struggle” panel, the Predictions page lists them all, and the What-if lab lets the teacher change marks and attendance to see how the probability responds. The rule explains the risk in plain words; the model can warn about a student who does not yet cross either threshold.

3.5 Application Modules

Module	Capability
Dashboard	Total students, average score, average attendance, students at risk, predicted to struggle
Students	Add, search, sort, filter by course and grade, at-risk filter, edit, delete, open report
Marks	Enter, edit and delete scores per student, exam and subject
Attendance	Record and view attendance per student
Courses	Maintain the courses students belong to
Predictions	Failure probability for every student from the regression model
What-if lab	Change inputs and recompute the prediction
Model	Information about the trained model
Import & export	Load and save data in bulk

Module	Capability
/metrics	Prometheus text-format metrics of the backend

Table 3.2: Modules of the application

4. Application Implementation

The React application has a persistent sidebar with four groups: Overview (Dashboard), Manage (Students, Marks, Attendance, Courses), Insights (Predictions, What-if lab, Model) and Data (Import & export). Each page is a separate component. The pages call the Flask API, which holds the business rules, so the risk rule and grade calculation are identical wherever they are displayed.

4.1 Dashboard

The dashboard (Figure 4.1) opens with four summary cards: 5 students, an average score of 83.3 out of 100, an average attendance of 71.4 % and 4 students at risk. Below them the "Students at risk" panel lists each such student with the roll number, course and the reason for the flag; in the sample data all four are listed for attendance below 75 %. Every row has a Report button that opens that student's performance report. A second panel, "Predicted to struggle", shows the highest failure probabilities from the regression model.

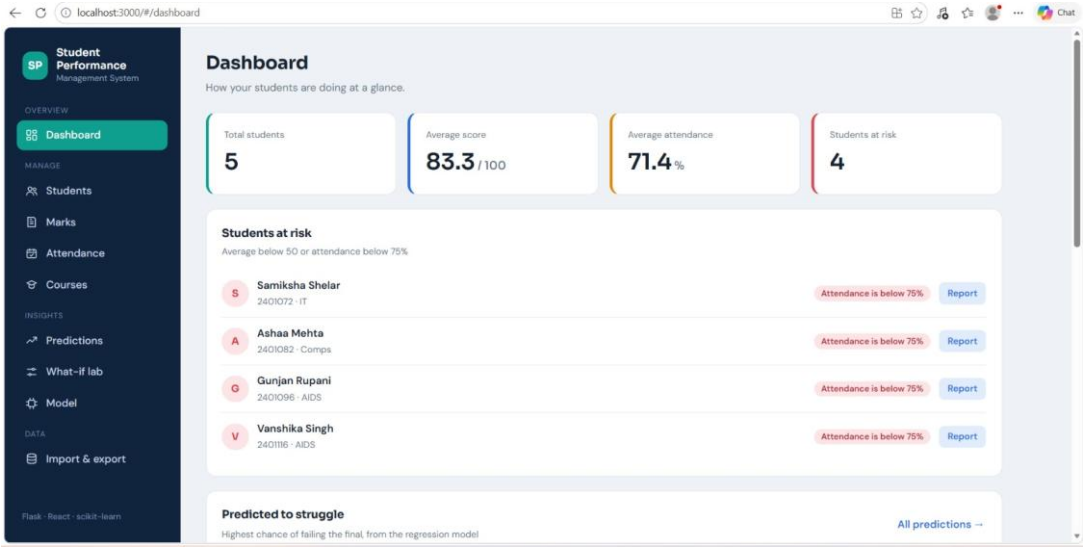


Figure 4.1: Dashboard with summary cards and students at risk

4.2 Student Management

The Students page (Figure 4.2) has an "Add a student" form requiring a unique roll number, full name and course, followed by a table of all students. The table can be searched, filtered by course and grade, restricted to at-risk students and sorted by clicking the column headers. Each row shows the average, the grade as a coloured badge, the attendance percentage and a status of "At risk" or "On track", with Report, Edit and Delete actions. For example, Krisha Samtani (2401099, AIDS) has an average of 88, grade A and 100 % attendance and is on track, whereas Samiksha Shelar (2401072, IT) has an average of 76 but only 50 % attendance and is flagged at risk.

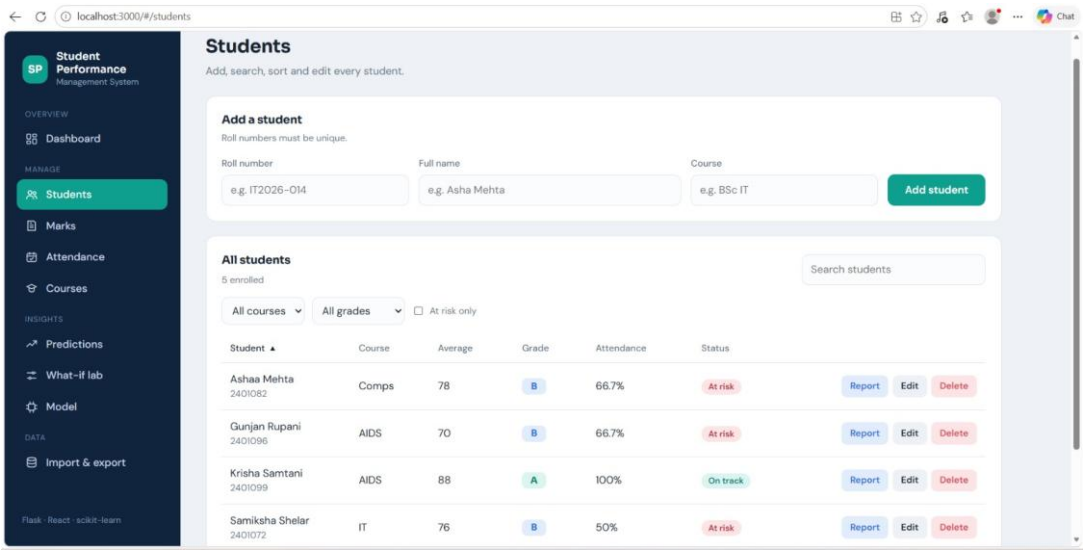


Figure 4.2: Students page with search, filters and status

4.3 Marks Management

On the Marks page (Figure 4.3) the teacher selects a student, an exam (for example Mid-term), types the subject and a score between 0 and 100, and saves. The right-hand panel lists the selected student's marks with Edit and Delete buttons and shows the average and grade at the top. In the figure Krisha Samtani has 93 in ASD & D and 83 in Stats, giving an average of 88 and grade A. Because entering the same subject and exam again updates the score, a wrongly typed mark is corrected by simply saving the right one.

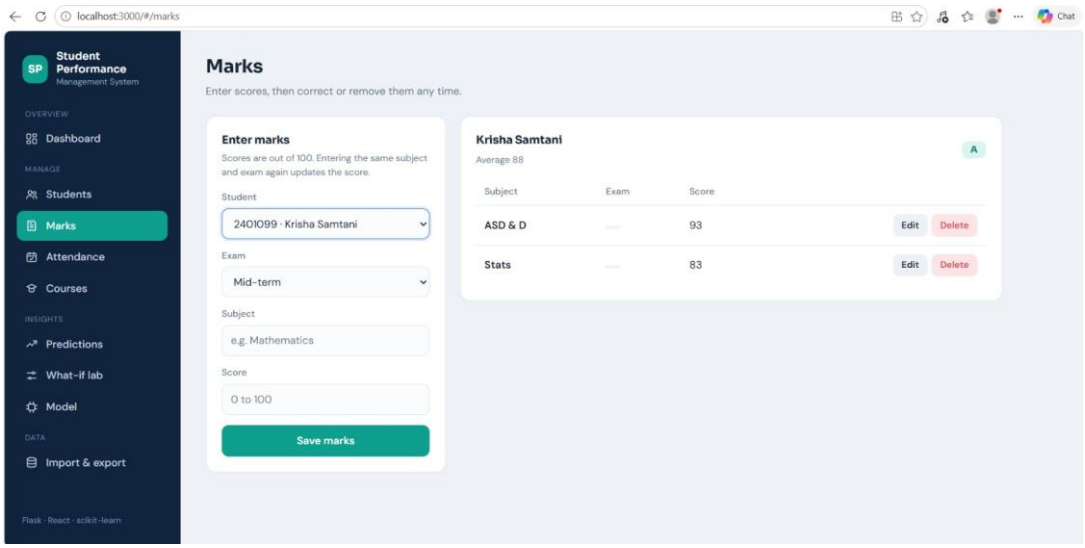


Figure 4.3: Marks page with the entry form and the student's score list

4.4 Attendance, Courses, Reports and Data

The Attendance page records the attendance of each student, and the percentage feeds both the Students table and the risk rule. The Courses page maintains the course list used by the student form and filters. The report module (report_pdf.py) creates a performance report for one student containing the marks, average, grade and attendance. The Import & export page loads data in bulk and saves it for backup or use in other tools.

4.5 Predictions and What-if Lab

The machine-learning code lives in the ml directory and is tested by test_ml_and_data.py. The model is trained on the stored marks and attendance and produces, for each student, the chance of failing the final examination. The Predictions page ranks students by this chance, the Model page explains how the model was trained, and the What-if lab lets a teacher answer questions such as “if this student raises attendance from 66 % to 85 %, how much does the chance of failure drop?” without changing any stored data.

4.6 Instrumentation

The backend exposes a /metrics endpoint in Prometheus text format. Prometheus scrapes it, and the up metric, which is 1 while the scrape succeeds, is used to show that the service is alive (Section 5.6). The endpoint is what turns the application from a black box into something the monitoring tools can observe.

5. DevOps Implementation

This chapter describes how each lab experiment was applied. Figure 5.1 summarises the delivery pipeline: a change is made on a branch, built and tested by Jenkins, merged into main, deployed as containers with Docker Compose and then monitored. The planning in Jira (Experiment 6) is described in Chapter 2.

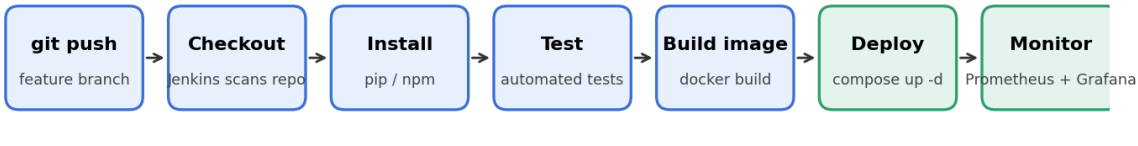


Figure 5.1: Delivery pipeline from push to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

The source code is kept in the GitHub repository **krishasamtani/Student-Performance-Management-System**. The repository contains the Flask application (app.py, the app folder with its tests), the machine-learning code (ml), the React frontend (spms-frontend), the Dockerfile, docker-compose.yml, prometheus.yml, report_pdf.py, requirements.txt and the README. The Git operations of Experiment 1 were used throughout:

```
git clone https://github.com/krishasamtani/Student-Performance-Management-
System.git
git checkout -b feature/SPMS-<n>-<short-name>   # one branch per Jira work item
git add .
git commit -m "SPMS-<n> <what changed>"
git push origin feature/SPMS-<n>-<short-name>
git checkout main
git merge feature/SPMS-<n>-<short-name>
git log --oneline --graph
```

Jira's Development tab (Figure 5.2) shows the repository connected to the space, which is what links work items to branches and commits.

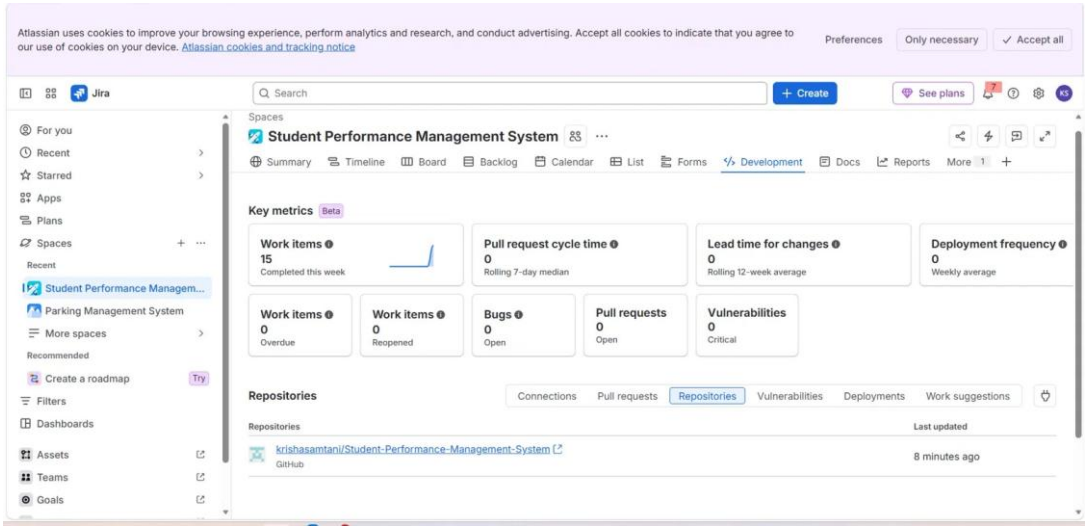


Figure 5.2: Jira Development tab with the GitHub repository connected

5.2 Containerisation with Docker (Experiment 2)

The Flask backend is packaged with a Dockerfile in the project root. The image installs the Python dependencies from requirements.txt, copies the application and the ml code, and starts the Flask server on port 5000. Keeping the database outside the image, in a directory mounted at runtime, means the image can be rebuilt freely without losing records. Docker Desktop (Figure 5.4) shows the resulting container, named spms-backend, running and published on port 5000.

```
docker build -t student-performance-management-system-backend .
docker run -d -p 5000:5000 -v ./data:/app/data --name spms-backend student-
performance-management-system-backend
docker ps
```

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

A single docker-compose.yml (Compose file version 3.8) describes the stack. The backend service is built from the local Dockerfile, named spms-backend, published on 5000:5000, receives the database location through the environment variable DB_PATH=/app/data/spms.db, mounts ./data to /app/data and is set to restart always. The prometheus service uses the prom/prometheus image, is published on 9090 and mounts the project's prometheus.yml read-only. The grafana service uses grafana/grafana and is published on host port 3003, because port 3000 on the host is already used by the React development server.

```
services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: spms-backend
    ports:
      - "5000:5000"
    environment:
      - DB_PATH=/app/data/spms.db
    volumes:
      - ./data:/app/data
    restart: always

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
  # grafana service: image grafana/grafana, host port 3003 -> 3000
```

The whole stack is started from the VS Code terminal with one command, `docker compose up -d --build`. Figure 5.3 shows the output: the backend image is built, and the prometheus, grafana and spms-backend containers are created and running. Figure 5.4 shows the same stack in Docker Desktop under the project name student-performance-management-system, where the Grafana log displays its database migrations completing successfully on first start.

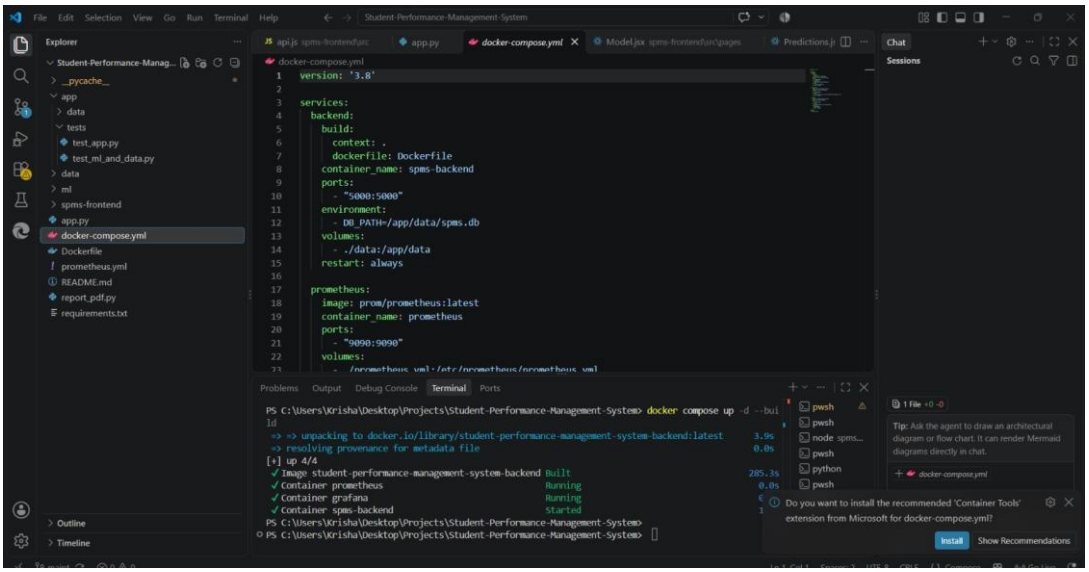


Figure 5.3: docker-compose.yml and the docker compose up output in VS Code

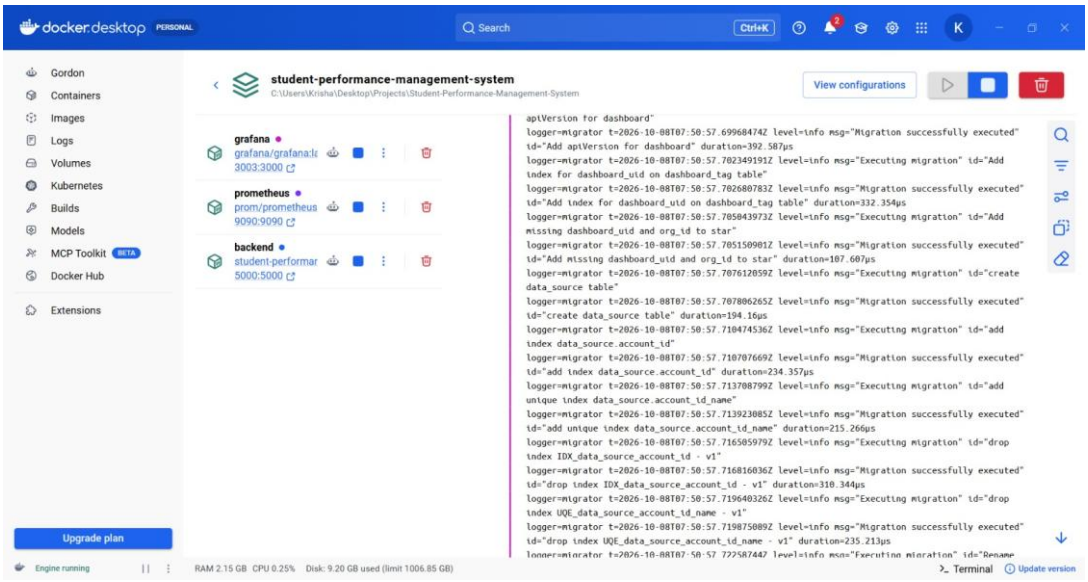


Figure 5.4: Docker Desktop showing the grafana, prometheus and backend containers

5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins runs on the developer machine at port 8080 and has a job named **Student-Performance-Management-System** that pulls the repository from GitHub. On every build it checks out the code, installs the dependencies, runs the automated tests (test_app.py for the API and test_ml_and_data.py for the data and machine-learning code) and builds the Docker image, so a change that breaks the tests or the image build is detected before it is deployed.

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Install')  { steps { /* install dependencies */ } }
    stage('Test')     { steps { /* run the automated tests */ } }
    stage('Build')    { steps { /* docker compose build */ } }
    stage('Deploy')   { steps { /* docker compose up -d */ } }
  }
}
```

The listing above shows the structure of the declarative pipeline: checkout, install, test, build and deploy stages run in sequence.

5.5 Automated Deployment (Experiment 5)

The final stage of the pipeline deploys the project by running Docker Compose on the same host, so the containers are recreated from the image that was just built. The Jenkins job page (Figure 5.5) shows the result: the build history lists build #1 at 12:34 AM and build #10 at 12:56 AM on 8 October 2026, and the permalinks show build #10 as the last build, the last stable build, the last successful build and the last completed build. The green tick on the job confirms that the latest pipeline run, including deployment, succeeded.

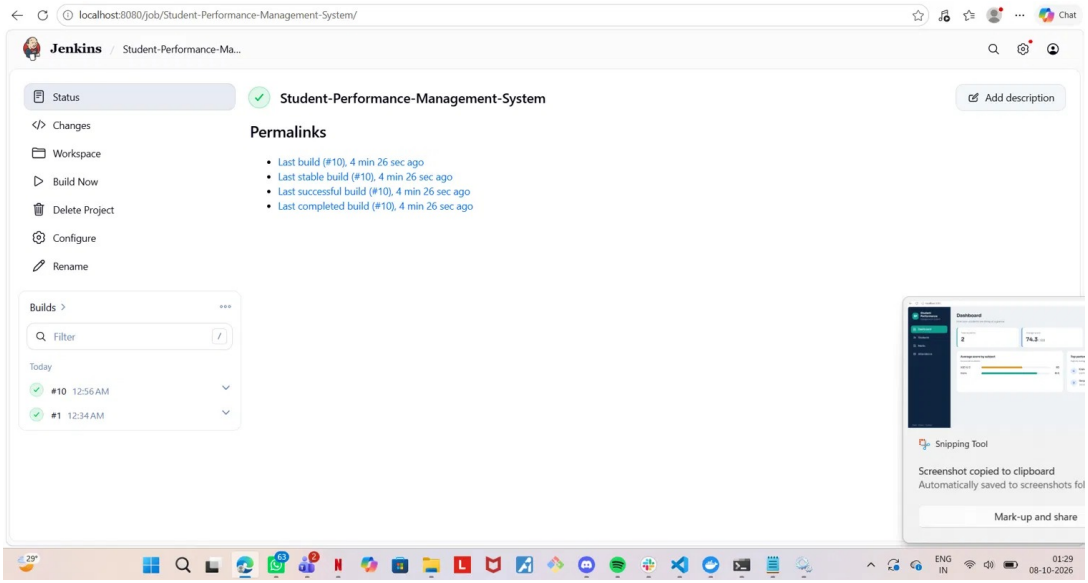


Figure 5.5: Jenkins job – last build #10 stable and successful

5.6 Monitoring with Prometheus and Grafana (Experiment 7)

Experiment 7 introduced monitoring as the practice of observing a service after it is deployed. In this project Prometheus and Grafana were used, both running as containers in the Compose stack. The Prometheus configuration (prometheus.yml) defines a job named **spms-backend** that scrapes http://spms-backend:5000/metrics; the address uses the Compose service name rather than localhost because Prometheus runs in its own container. The Targets page of Prometheus (Figure 5.6) shows this endpoint as UP, the instance label spms-backend:5000, a scrape duration of about 5 ms, and "1 / 1 up" for the job.

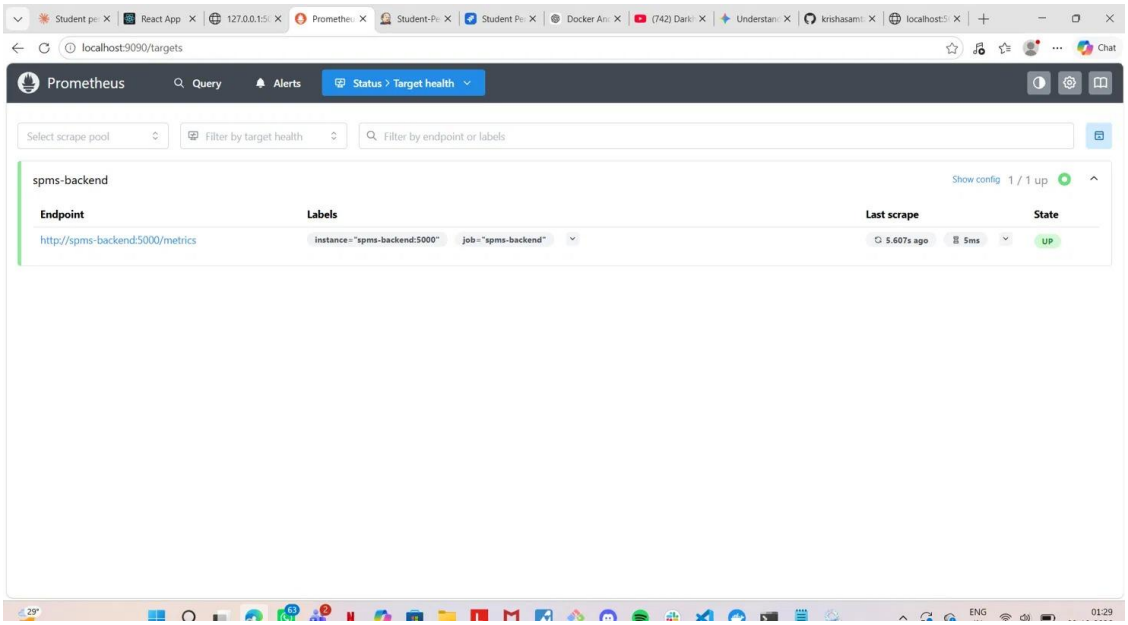


Figure 5.6: Prometheus target health – spms-backend is UP

Grafana, reachable on port 3003, uses Prometheus as its data source. In Figure 5.7 a new time-series panel was created from the query `up`, with a refresh interval of 15 seconds over the last five minutes. The graph is a flat line at 1 for the series {instance="spms-backend:5000",

job="spms-backend"}, which means that every scrape in the period succeeded and the backend was available throughout. If the container were stopped, the value would drop to 0, which is the condition on which an alert would be raised.

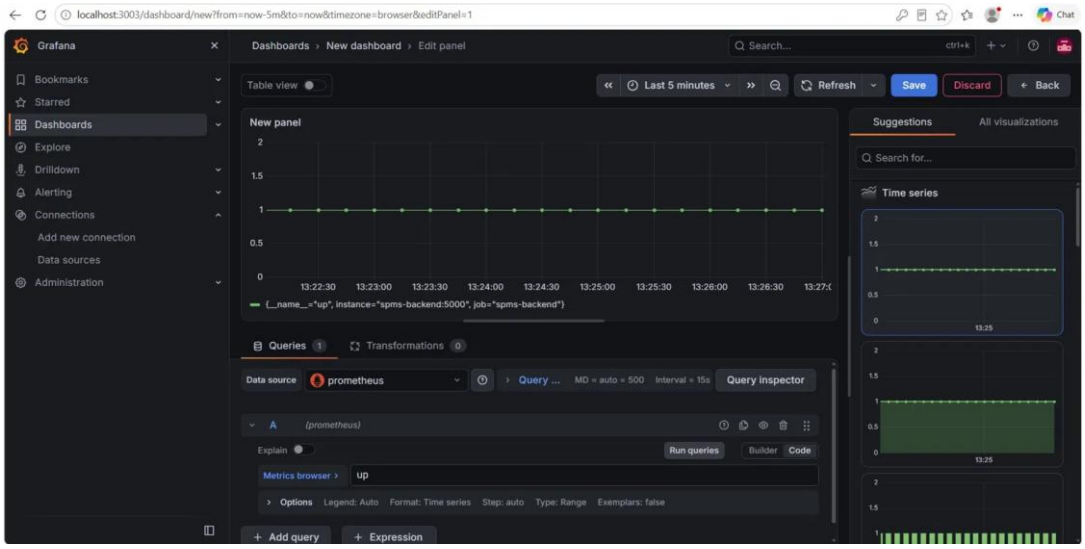


Figure 5.7: Grafana panel plotting the up metric of the backend

6. Testing

6.1 Automated Tests

The repository contains two test modules in app/tests. **test_app.py** exercises the Flask application, covering the student, marks, attendance and report operations through the API. **test_ml_and_data.py** tests the data handling and the machine-learning code. Because the tests run without the browser, they are executed in seconds by Jenkins on every build.

6.2 Functional Test Cases

No.	Test case	Expected result	Result
1	Add a student with a new roll number	Student appears in the list; total on dashboard increases	Pass
2	Add a student with an existing roll number	Rejected – roll numbers must be unique	Pass
3	Enter a score of 93 for ASD & D (Mid-term)	Score saved; average and grade recalculated	Pass
4	Enter a score outside 0 to 100	Rejected with a validation message	Pass
5	Enter the same subject and exam again	Existing score is updated, no duplicate row	Pass
6	Student with attendance below 75 %	Status “At risk” and listed on dashboard	Pass
7	Tick “At risk only” and filter by course	Table shows only matching students	Pass
8	Open Report for a student	Performance report is generated	Pass
9	Open /metrics on the backend	Prometheus text-format metrics returned	Pass

Table 6.1: Functional test cases

6.3 Deployment and Monitoring Checks

After each deployment the containers were checked with docker compose ps and Docker Desktop, the application was opened in the browser, and the Prometheus Targets page was inspected to confirm that spms-backend was UP. The Grafana up graph then provided continuous evidence that the service remained available.

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Student management with search and filters	Delivered (SPMS-1); Figure 4.2
Marks, average and grade	Delivered (SPMS-2); Figure 4.3
Attendance and at-risk rule	Delivered (SPMS-3); Figure 4.1
Prediction and what-if analysis	Delivered; Predictions, What-if lab and Model pages
Performance reports	Delivered (SPMS-4, SPMS-11)
Scrum planning in Jira	5 epics, 10 stories, SPMS Sprint 2, all 15 items Done
Source code on GitHub linked to Jira	Repository connected in the Development tab
Containerised stack, one command	Backend, Prometheus and Grafana started by docker compose up -d --build (SPMS-5, 12, 13)
Automated build and deployment	Jenkins build #10 stable and successful
Monitoring	Prometheus target UP; Grafana panel on the up metric

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The practices reinforced each other. The Jira work item defined what was to be built, GitHub kept the code, Jenkins turned each change into a tested and deployed build, Docker Compose guaranteed that the deployed stack was the stack the developers ran, and Prometheus and Grafana showed that the deployed service stayed healthy. The Jira Reports page, which shows all fifteen work items finished, and the Jenkins page, which shows a stable build, are two views of the same completed iteration.

7.3 Challenges and Resolutions

Challenge	Resolution
Port 3000 is used by the React development server, which clashes with Grafana's default port	Grafana container port 3000 published on host port 3003
Prometheus cannot reach the backend through localhost because it runs in its own container	Scrape target uses the Compose service name spms-backend:5000
Database must survive container rebuilds	Database directory mounted as a volume (./data to /app/data) and DB_PATH set by environment variable
Jenkins and the application both run on the same machine	Jenkins on port 8080, backend on 5000, Prometheus on 9090, Grafana on 3003, React on 3000
Free Jira trial period is limited	Reports and timeline screenshots were captured as evidence while the trial was active

Table 7.2: Challenges met and how they were resolved

7.4 Limitations

- Jenkins, the application and the monitoring stack share one host, so a host failure would remove both the system and the means of observing it.
- Only the availability metric (up) is graphed; business metrics such as the number of students at risk are not yet exported to Grafana.
- Prometheus alerts are not routed to email or chat because no Alertmanager is deployed.
- SQLite is adequate for a department but not for many simultaneous writers.
- The application has no login, so any user who can reach it can edit records.

8. Conclusion and Future Scope

8.1 Conclusion

The Student Performance Management System meets its functional goal. A teacher can maintain students, courses, marks and attendance in one place, see immediately which students are at risk and why, and consult a prediction of who is likely to fail the final examination. More importantly for this course, the project applied the lab experiments to one system rather than to isolated exercises.

Jira gave the work a structure of epics, stories and a sprint, and its reports show all fifteen items completed. GitHub stored the code and was linked back to Jira. Docker and Docker Compose made the environment reproducible, so that the backend, Prometheus and Grafana start together with a single command. Jenkins automated the build and deployment, ending with a stable, successful build, and Prometheus and Grafana showed that the deployed backend was up. Each practice was most useful for what it made possible in the next one.

8.2 Future Scope

- User accounts with separate teacher, administrator and student roles.
- Export of business metrics (students at risk, average attendance) to Prometheus and a Grafana dashboard with alert rules.
- Alertmanager to send alerts by email or chat when the backend goes down.
- Email or SMS notification to students and parents when attendance falls below 75 %.
- Migration from SQLite to PostgreSQL or MySQL and deployment to a Kubernetes cluster.
- Retraining the model automatically each term, tracked with MLflow, as in the MLOps part of the course.

REFERENCES

1. Thadomal Shahani Engineering College, Department of Artificial Intelligence & Data Science, Lab Manual: Agile Software Development & DevOps Lab (2165119), NEP Scheme, 2026–2027.
2. Git Documentation, https://git-scm.com/doc
3. Docker Documentation, https://docs.docker.com
4. Docker Compose Documentation, https://docs.docker.com/compose
5. Jenkins User Documentation, https://www.jenkins.io/doc
6. Atlassian Jira Software Documentation, https://support.atlassian.com/jira-software-cloud
7. Prometheus Documentation, https://prometheus.io/docs
8. Grafana Documentation, https://grafana.com/docs
9. Flask Documentation, https://flask.palletsprojects.com
10. scikit-learn User Guide, https://scikit-learn.org/stable/user_guide.html
11. React Documentation, https://react.dev

APPENDIX A: Running the Project

Start the full stack

```
git clone https://github.com/krishasamtani/Student-Performance-Management-System.git
cd Student-Performance-Management-System
docker compose up -d --build
docker compose ps
```

Addresses

Service	Address
React frontend	http://localhost:3000
Flask backend and /metrics	http://localhost:5000 · http://localhost:5000/metrics
Prometheus	http://localhost:9090 (Status › Target health)
Grafana	http://localhost:3003
Jenkins	http://localhost:8080

Table A.1: Service addresses

Run the tests

```
pip install -r requirements.txt
pytest app/tests
```

Stop the stack

```
docker compose down
```